



# Nuevos paradigmas de Base de Datos

## 1. Guía completa para configurar un entorno de desarrollo con Docker desde Visual Studio Code

En esta guía configuraremos un entorno de desarrollo utilizando Docker y Visual Studio Code. Se instalarán los siguientes servicios mediante contenedores Docker:

- **MySQL** (con Dockerfile personalizado para cargar archivos locales del proyecto)
- **Apache Airflow**
- **MongoDB**

Todo se gestionará mediante docker-compose y se ejecutará desde Visual Studio Code.

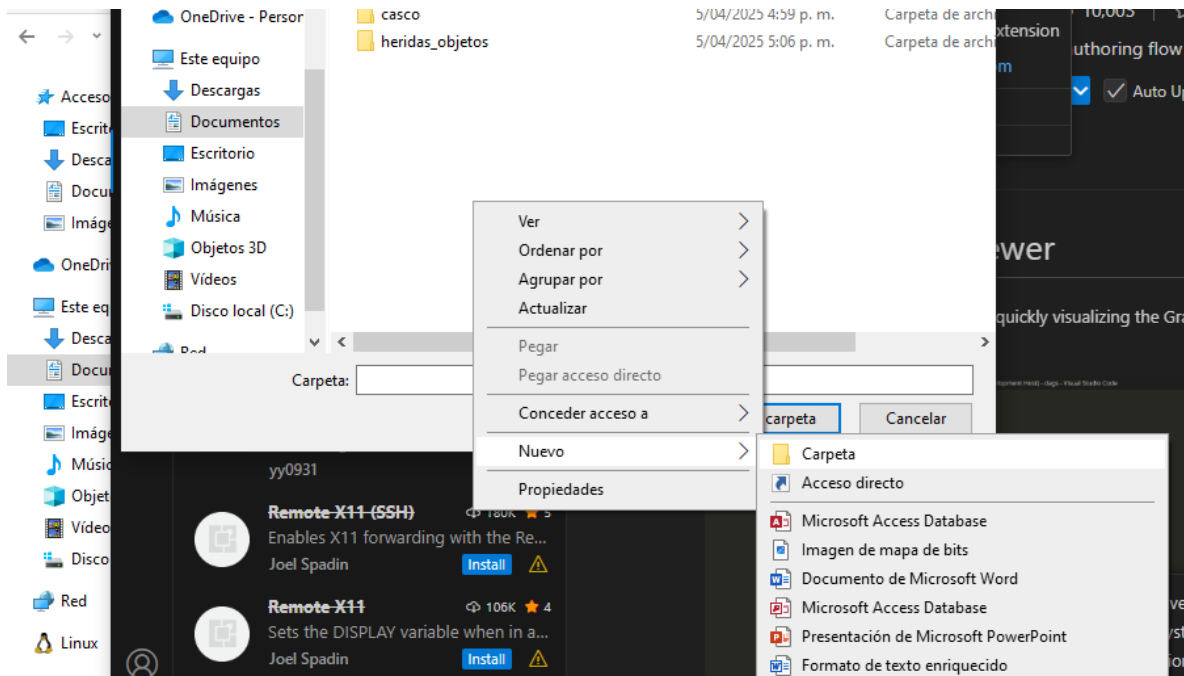
## 2. Requisitos previos

Asegúrate de tener instalados los siguientes programas y extensiones:

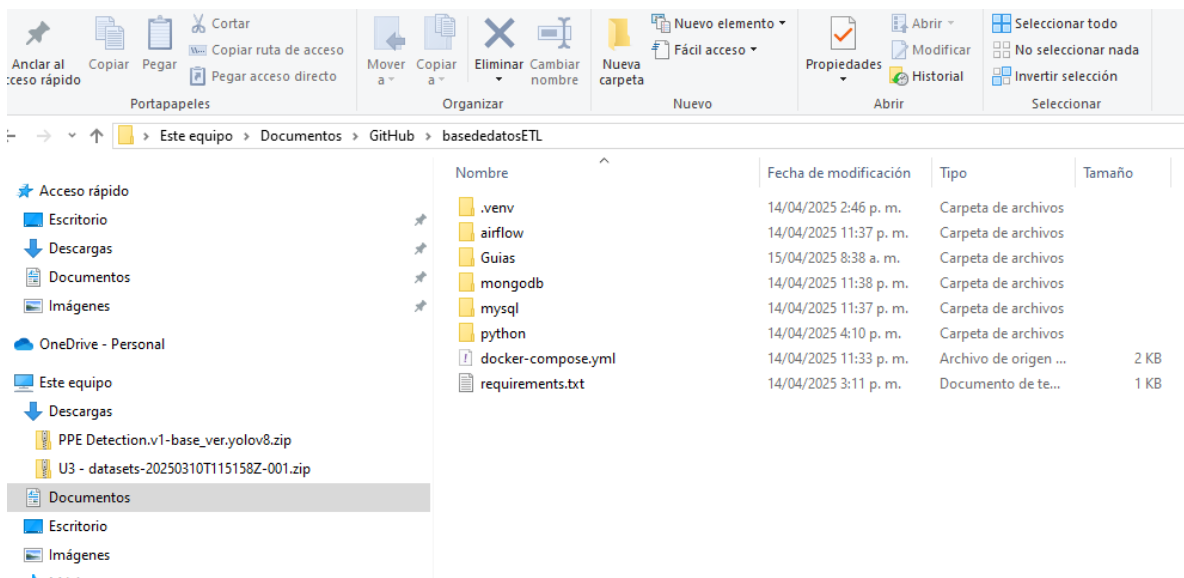
- Docker Desktop
- Visual Studio Code
- Extensiones para VS Code:
  - Docker
  - Remote - Containers
  - Dev Containers (opcional)
  - SQLTools (para conectarse a MySQL)
- Python y Jupyter si se trabajará con notebooks

## 3. Estructura del proyecto

Crea la carpeta paradigmas y organiza el proyecto de acuerdo con la siguiente estructura, donde pondremos todos nuestros talleres de la Unidad 3 del curso.



La carpeta al finalizar esta guía quedará así:



Descarga los siguientes archivos de GitHub y guárdelos en la carpeta de mysql del proyecto:

### 3.1 Descarga de archivos necesarios

La opción recomendada es descargar todas las carpetas y archivos desde el siguiente github

<https://github.com/adiac/a/basededatosETL/raw/refs/heads/main/paradigmas.zip>

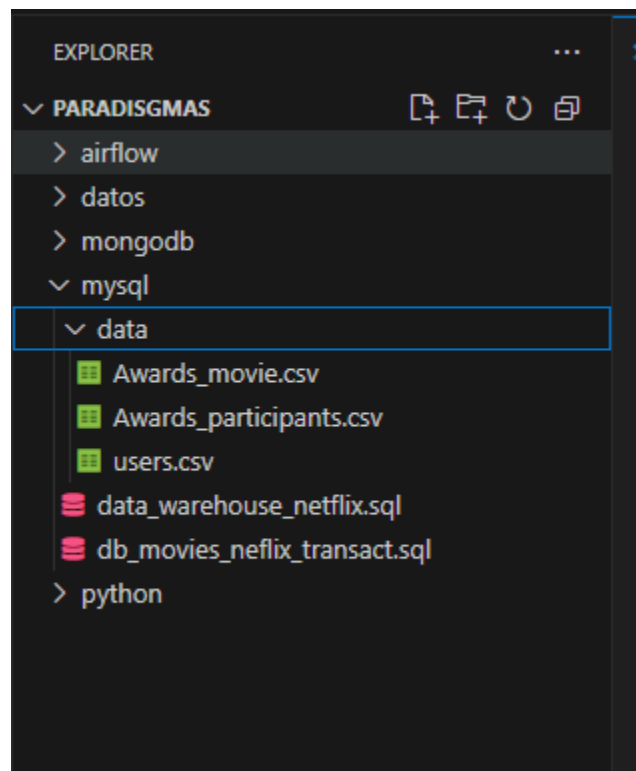
Otra opción no recomendada es descargar los archivos de la siguiente forma:

Descarga los siguientes archivos SQL y colóquelos en la carpeta mysql:

- [data\\_warehouse\\_netflix.sql](#)
- [db\\_movies\\_neflix\\_transact.sql](#)
- [o-create-airflow-db.sql](#)

Luego descarga los siguientes archivos CSV y colóquelos en mysql/data:

- [Awards\\_movie.csv](#)
- [Awards\\_participants.csv](#)
- [users.csv](#)





#### 4. Crear docker-compose.yml en la raíz del proyecto

En VS Code, crea un archivo docker-compose.yml en la carpeta paradigmas con el siguiente contenido:

```
services:
  mysql:
    image: mysql:8.0
    restart: always
    ports:
      - '3310:3306'
    environment:
      - MYSQL_ROOT_PASSWORD=root
    volumes:
      - ./mysql/o-create-airflow-db.sql:/docker-entrypoint-initdb.d/o-create-airflow-db.sql
      - ./mysql/db_movies_netflix_transact.sql:/docker-entrypoint-initdb.d/1-db_movies.sql
      - ./mysql/data_warehouse_netflix.sql:/docker-entrypoint-initdb.d/2-dw_netflix.sql
      - ./mysql/data:/data

  mongodb:
    image: mongo:6.0
    restart: always
    ports:
      - "27017:27017"
    volumes:
      - mongodata:/data/db
      - ./mongodb/data:/data/init

  airflow:
    build:
      context: ./airflow
      dockerfile: Dockerfile
    container_name: airflow
    restart: always
    depends_on:
      - mysql
    ports:
      - "8080:8080"
    environment:
      - AIRFLOW__CORE__EXECUTOR=SequentialExecutor
      - AIRFLOW__CORE__LOAD_EXAMPLES=False
```

```

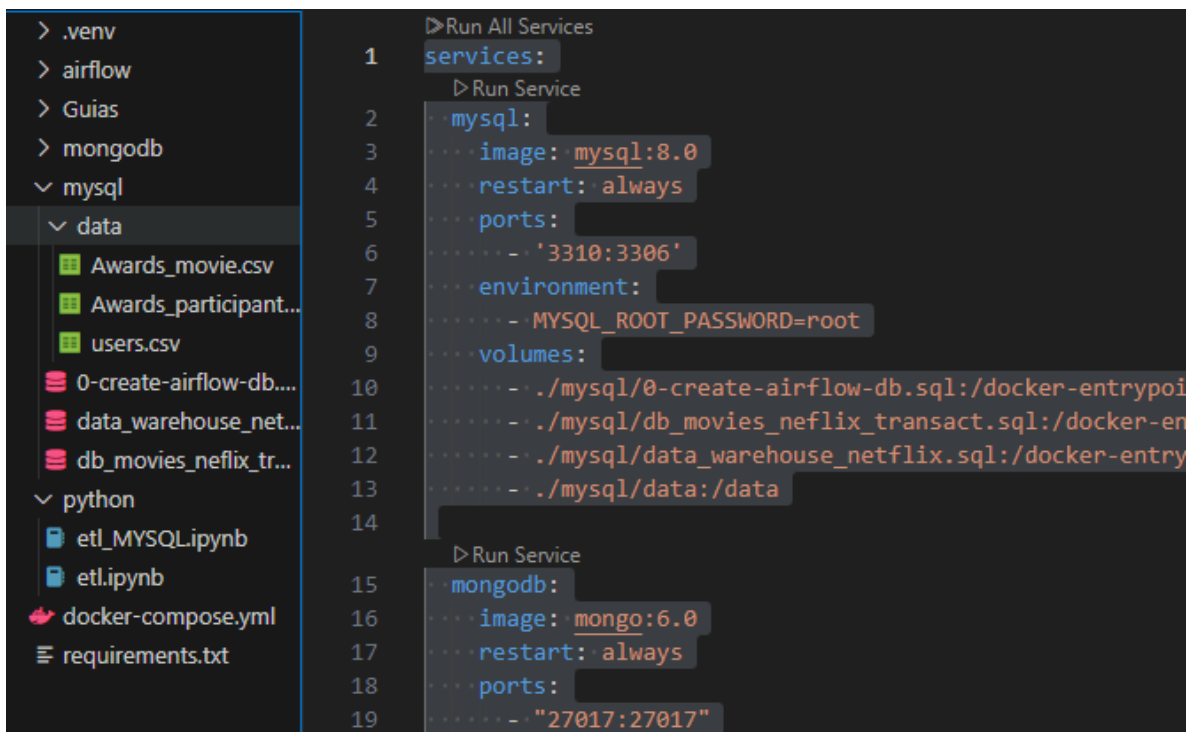
-
AIRFLOW__DATABASE__SQL_ALCHEMY_CONN=mysql+pymysql://root:root
@mysql:3306/airflow
volumes:
- ./airflow/dags:/opt/airflow/dags
- ./airflow/logs:/opt/airflow/logs
- ./airflow/plugins:/opt/airflow/plugins
- ./mysql/data:/data
command: >
  bash -c "airflow db init && airflow users create
  --username admin --password admin --firstname Admin --lastname User
  --role Admin --email admin@example.com && airflow webserver"

volumes:
mongodata:
  dockerfile: Dockerfile
  container_name: airflow
  restart: always
  ports:
  - "8080:8080"
  environment:
  - AIRFLOW__CORE__EXECUTOR=SequentialExecutor
  - AIRFLOW__CORE__LOAD_EXAMPLES=False
  - -
AIRFLOW__DATABASE__SQL_ALCHEMY_CONN=mysql+pymysql://root:root
@mysql:3306/airflow
- AIRFLOW__DATABASE__LOAD_DEFAULT_CONNECTIONS=False
depends_on:
- mysql
volumes:
- ./airflow/dags:/opt/airflow/dags
- ./airflow/logs:/opt/airflow/logs
- ./airflow/plugins:/opt/airflow/plugins
- ./mysql/datos:/datos
command: >
  bash -c "airflow db init && airflow users create
  --username admin --password admin --firstname Admin --lastname User
  --role Admin --email admin@example.com && airflow webserver"

volumes:
mongodata:

```

Lo puedes descargar de [docker-compose.yml](#) y guardar en el raíz de "paradigma"



```
> .venv
> airflow
> Guias
> mongodb
v mysql
  v data
    Awards_movie.csv
    Awards_participant...
    users.csv
    0-create-airflow-db....
    data_warehouse_net...
    db_movies_neflix_tr...
  v python
    etl_MYSQLipynb
    etl.ipynb
    docker-compose.yml
    requirements.txt

1  services:
2    mysql:
3      image: mysql:8.0
4      restart: always
5      ports:
6        - '3310:3306'
7      environment:
8        - MYSQL_ROOT_PASSWORD=root
9      volumes:
10       - ./mysql/0-create-airflow-db.sql:/docker-entrypoint-initdb.d/0-create-airflow-db.sql
11       - ./mysql/db_movies_neflix_transactions.sql:/docker-entrypoint-initdb.d/db_movies_neflix_transactions.sql
12       - ./mysql/data_warehouse_netflix.sql:/docker-entrypoint-initdb.d/data_warehouse_netflix.sql
13       - ./mysql/data:/data
14
15   mongodb:
16     image: mongo:6.0
17     restart: always
18     ports:
19       - "27017:27017"
```

## 5. Crear Dockerfile personalizado para Airflow

Crea el archivo Dockerfile dentro de airflow/ con el siguiente contenido:

**Airflow usa una imagen oficial** que no trae ni pymysql ni pandas por defecto. Por lo tanto debemos crear una **Dockerfile personalizada** para eso. Crear un archivo Dockerfile sin extensión como un txt.

```
RUN apt-get update && apt-get install -y --no-install-recommends \
  build-essential \
  gcc \
  libpq-dev \
  libffi-dev \
  libssl-dev \
  default-libmysqlclient-dev \
  && apt-get clean \
  && rm -rf /var/lib/apt/lists/*

# Etapa 2: cambiar a usuario airflow para instalar paquetes Python
USER airflow

RUN pip install --no-cache-dir pandas pymysql
```

O lo puedes descargar de [Dockerfile](#) y lo guardas dentro de la carpeta airflow

## 6. Crear archivo de inicialización de base de datos para Airflow

Si no has descargado el Archivo 0-create-airflow-db.sql puedes crearlo como texto:

En la carpeta ./mysql/0-create-airflow-db.sql, este debe tener solo esta línea:

```
CREATE DATABASE IF NOT EXISTS airflow;
```

## 7. Instalar dependencias de Python localmente

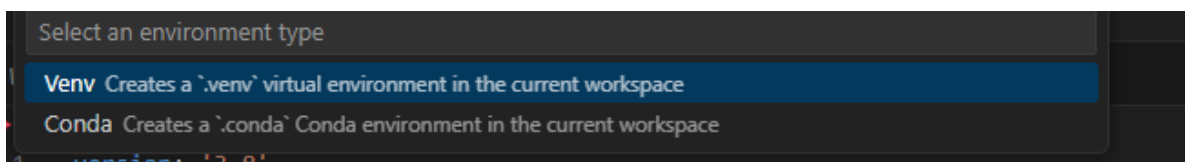
Descarga [requirements.txt](#) al directorio raíz del proyecto. (si descargaste el zip de github no es necesario descargarlo)

Crea un entorno virtual desde VS Code:

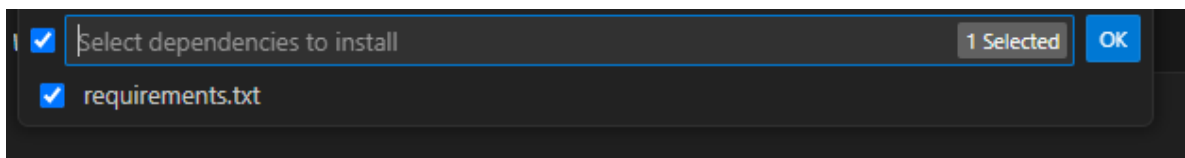
1. Presiona CTRL+Shift+P
2. Selecciona Python: Create Environment
3. Selecciona Venv y el interprete Python 3.11
4. Apunta al archivo requirements.txt y confirma con OK

De este modo.

Python: Create environment (Si no aparece es que no has instalado Python Extension Pack)



Selecciona Venv en intérprete de Python. En este caso el 3.11.xx y selecciona el archivo requirements.txt para que se instalen automáticamente todas las bibliotecas y dependencias. Y luego clic en O.k. Si tiene error verifica permisos o conexión a internet.

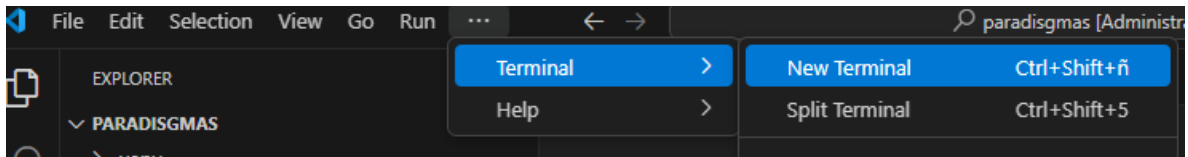




## 8. Activar el entorno virtual

Activa la terminal integrada con:

- CTRL+Shift+ñ
- O desde el menú superior: **Terminal > Nueva Terminal**



## 9. Verificar Docker

Ejecuta en la terminal:

```
docker --version  
docker-compose --version
```

```
PS C:\Users\Administrador\Documents\GitHub\paradisgmas> docker --version  
Docker version 28.0.4, build b8034c0  
PS C:\Users\Administrador\Documents\GitHub\paradisgmas> docker-compose --version  
Docker Compose version v2.34.0-desktop.1  
PS C:\Users\Administrador\Documents\GitHub\paradisgmas> 
```

## 10. Ejecutar el entorno

Levanta los contenedores:

Ejecutar el archivo docker-compose.yml desde Visual Studio Code (VS Code) usando el terminal integrado.

```
docker compose up -d
```

```
PS C:\Users\Administrador\Documents\GitHub\paradigmamas> docker-compose up -d
time="2025-04-14T11:57:55-05:00" level=warning msg="C:\\Users\\Administrador\\Documents\\GitHub\\paradigmamas\\docker-compose.yml:
ute `version` is obsolete, it will be ignored, please remove it to avoid potential confusion"
[+] Running 32/44
- airflow [ ] 78.94MB / 363.1MB Pulling
- mysql [ ] 76.11MB / 230.9MB Pulling
- mongod [ ] 79.87MB / 261.2MB Pulling
```

Si hay cambios en docker-compose.yml o Dockerfile: Para detener usa ***docker compose down*** y para usar ***docker compose up*** y si hiciste cambios en tu docker-compose.yml o en el Dockerfile y quieres reconstruir la imagen, el comando ideal es: ***docker compose up --build***

## 11. Iniciar el scheduler de Airflow

### Ingresar al contenedor:

Aunque tu contenedor está corriendo, el scheduler podría no estar arrancando automáticamente. Para asegurarte de que esté en funcionamiento, ejecuta el siguiente comando dentro del contenedor de Airflow:

```
docker exec -it airflow bash
```

Esto te llevará al contenedor en modo interactivo. Una vez dentro, ejecuta:

```
airflow scheduler
```

Esto debería iniciar el scheduler de Airflow en segundo plano.

## 12. Verificar contenedores en ejecución

```
docker ps
```

```
PS C:\Users\Administrador\Documents\GitHub\paradigmamas> docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS        PORTS                               NAMES
e6f7d542de3e   mysql:8.0                "docker-entrypoint.s..." About a minute ago Up About a minute   33060/tcp      paradigmamas-mysql-1
64655672df22   apache/airflow:2.8.1-python3.11  "/usr/bin/dumb-init ..." About a minute ago Up About a minute   0.0.0.0:8080->8080/tcp airflow
878cb91daf30   mongo:6.0                "docker-entrypoint.s..." About a minute ago Up About a minute   0.0.0.0:27017->27017/tcp paradigmamas-mongodb-1
```

## 13. Conectarse al contenedor MySQL

El contenedor **MySQL** está corriendo en el puerto 3310, y para acceder a él, puedes usar el siguiente comando.

1. Conéctate al contenedor MySQL con el terminal de Vsc:

```
docker exec -it paradigmas-mysql-1 bash
```

```
PS C:\Users\Administrador\Documents\GitHub\paradigmas> docker exec -it paradigmas-mysql-1 bash
bash-5.1#
```

2. Inicie MySQL:

Una vez dentro del contenedor, ejecuta el cliente de MySQL: Ingresa la contraseña **root** (si configuraste esa contraseña en el docker-compose.yml).

```
mysql -u root -p
```

```
bash-5.1# mysql -u root -p
Enter password:
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 11
Server version: 8.0.41 MySQL Community Server - GPL

Copyright (c) 2000, 2025, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.
```

3. Luego:

Ahora ya estás dentro de la interfaz de MySQL. Puedes listar las bases de datos:

```
SHOW DATABASES;
```

```
+-----+
| airflow
| db_movies_netflix_transact
| dw_netflix
| information_schema
| mysql
| performance_schema
| sys
+-----+
7 rows in set (0.00 sec)
```

Para trabajar con una base de datos, usa:

```
use db_movies_netflix_transact;
```

```
mysql> use db_movies_netflix_transact
Reading table information for completion of table and column names
You can turn off this feature to get a quicker startup with -A

Database changed
```

Luego, puedes listar las tablas con:

```
SHOW TABLES;
```

```
mysql> SHOW TABLES;
+-----+
| Tables_in_db_movies_netflix_transact |
+-----+
| gender                                |
| movie                                |
| movie_gender                          |
| participant                           |
| person                               |
+-----+
```

## 14. Conectarse al contenedor MongoDB

1. Accede al contenedor:

Para conectarte al contenedor **MongoDB**, usa este comando:

```
docker exec -it paradigmas-mongodb-1 bash
```

```
PS C:\Users\Administrador\Documents\GitHub\paradigmas> docker exec -it paradigmas-mongodb-1 bash
root@878cb91daf30:/#
```

2. Ejecute:

Una vez dentro, ejecuta el cliente de **MongoDB**:

```
mongosh
```

```
root@878cb91daf30:/# mongosh
Current Mongosh Log ID: 67fd489f8ced47e935d861df
Connecting to:      mongodb://127.0.0.1:27017/?directConnection=true&serverSelectionTimeoutMS=2000&appName=mongosh+2.5.0
Using MongoDB:      6.0.21
Using Mongosh:      2.5.0

For mongosh info see: https://www.mongodb.com/docs/mongosh-shell/
```

3. Dentro de **mongosh**, puedes listar las bases de datos:

```
show dbs
```

```
test> show dbs;
admin   40.00 KiB
config  12.00 KiB
local   40.00 KiB
test> |
```

Luego, si encuentras una base de datos que quieras explorar, ingresa a ella:

```
use local
```

```
test> use local
switched to db local
```

Puedes listar las colecciones dentro de esa base de datos con:

```
show collections
```

```
local> show collections;
startup_log
```